Commenting out body of deconstructor

```
rileyashok@Qubit:~/EECE2140/InClassWork11_Ashok$ valgrind --leak-check=full -s ./a
==95542== Memcheck, a memory error detector
==95542== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==95542== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==95542== Command: ./a
==95542==
[5: five)] -> [4: four)] -> [3: three] -> [2: two] -> [1: one] -> null
==95542==
==95542== HEAP SUMMARY:
==95542==     in use at exit: 240 bytes in 5 blocks
==95542==   total heap usage: 8 allocs, 3 frees, 75,008 bytes allocated
==95542==
==95542== 240 (48 direct, 192 indirect) bytes in 1 blocks are definitely lost in loss record 5 of 5
==95542==    at 0x4846FA3: operator new(unsigned long) (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==95542==    by 0x10A7F1: ??? (in /home/rileyashok/EECE2140/InClassWork11_Ashok/a)
==95542==    by 0x10A58D: ??? (in /home/rileyashok/EECE2140/InClassWork11_Ashok/a)
==95542==    by 0x4B361C9: (below main) (libc_start_call_main.h:58)
==95542==
==95542== LEAK SUMMARY:
==95542==    definitely lost: 48 bytes in 1 blocks
==95542==    indirectly lost: 192 bytes in 4 blocks
==95542==      possibly lost: 0 bytes in 0 blocks
==95542==    still reachable: 0 bytes in 0 blocks
==95542==         suppressed: 0 bytes in 0 blocks
==95542==
==95542== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
rileyashok@Qubit:~/EECE2140/InClassWork11_Ashok$
```

Deleting Node Twice

```
rileyashok@Qubit:~/EECE2140/InClassWork11_Ashok$ valgrind --leak-check=full -s ./a
==95615== Command: ./a
==95615==
[5: five)] -> [4: four)] -> [3: three] -> [2: two] -> [1: one] -> null
[5: five)] -> null
==95615==
==95615== HEAP SUMMARY:
==95615==     in use at exit: 64 bytes in 2 blocks
==95615==   total heap usage: 8 allocs, 6 frees, 75,008 bytes allocated
==95615==
==95615== 64 (16 direct, 48 indirect) bytes in 1 blocks are definitely lost in loss record 2 of 2
==95615==    at 0x4846FA3: operator new(unsigned long) (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==95615==    by 0x10A3EE: ??? (in /home/rileyashok/EECE2140/InClassWork11_Ashok/a)
==95615==    by 0x4B361C9: (below main) (libc_start_call_main.h:58)
==95615==
==95615== LEAK SUMMARY:
==95615==    definitely lost: 16 bytes in 1 blocks
==95615==    indirectly lost: 48 bytes in 1 blocks
==95615==      possibly lost: 0 bytes in 0 blocks
==95615==    still reachable: 0 bytes in 0 blocks
==95615==         suppressed: 0 bytes in 0 blocks
==95615==
==95615== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
rileyashok@Qubit:~/EECE2140/InClassWork11_Ashok$
```

Access node after deleting it

```
rileyashok@Qubit:~/EECE2140/InClassWork11_Ashok$ valgrind --leak-check=full -s ./a
==95686== Memcheck, a memory error detector
==95686== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==95686== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==95686== Command: ./a
==95686==
[5: five)] -> [4: four)] -> [3: three] -> [2: two] -> [1: one] -> null
[5: five)] -> null
==95686==
==95686== HEAP SUMMARY:
==95686==     in use at exit: 64 bytes in 2 blocks
==95686==   total heap usage: 8 allocs, 6 frees, 75,008 bytes allocated
==95686==
==95686== 64 (16 direct, 48 indirect) bytes in 1 blocks are definitely lost in loss record 2 of 2
==95686==    at 0x4846FA3: operator new(unsigned long) (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==95686==    by 0x10A3EE: ??? (in /home/rileyashok/EECE2140/InClassWork11_Ashok/a)
==95686==    by 0x4B361C9: (below main) (libc_start_call_main.h:58)
==95686==
==95686== LEAK SUMMARY:
==95686==    definitely lost: 16 bytes in 1 blocks
==95686==    indirectly lost: 48 bytes in 1 blocks
==95686==      possibly lost: 0 bytes in 0 blocks
==95686==    still reachable: 0 bytes in 0 blocks
==95686==         suppressed: 0 bytes in 0 blocks
==95686==
==95686== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
rileyashok@Qubit:~/EECE2140/InClassWork11_Ashok$
```

Write up:

Smart pointers can make code much shorter and cleaner, and also reduce the risk of memory leaks, double free, or dangling pointers. While in Manual you will need the Rule of Three, with smart pointers you don’t making it much easier.